

Tutoriel Prolog

1 Introduction

Prolog est un langage *déclaratif* (un peu comme SQL). Un programme Prolog est un ensemble de *faits* et de *règles* que l'on interroge avec des *requêtes* dans un interpréteur. Il n'y a pas de *for*, de *while* etc. Dans ce TP, nous avons à disposition deux moyens de coder :

1. l'interpréteur SWI-Prolog + un éditeur de texte (notepad, gedit, sublime text, atom, ...) pour écrire les programmes
2. SWISH (<https://swish.swi-prolog.org/>) qui est un outil en ligne.

Dans ce fichier, nous utiliserons la première solution. Nous allons donner quelques notions nécessaires pour mener à bien le TP :

- créer un programme Prolog simple
- quelques éléments de compréhension des listes

Pour ce faire, nous allons suivre un exemple fil rouge. Nous allons représenter les liens entre plusieurs personnes : Nicolas, Colin, Chloé, Chick, Alise, Isis, Jacquemort, Angel, Clémentine, Joël, Citroën et Noël. On s'intéresse à deux types de liens :

- l'amitié : on utilise une relation $ami(x, y)$ pour dire que x et y sont amis. Cette relation est symétrique, c'est-à-dire $ami(x, y)$ implique que $ami(y, x)$ est vrai aussi.
- l'amour : on utilise une relation $aime(x, y)$ pour dire que x aime y . Attention, ici x aime y n'implique pas forcément que y aime x .

On résume les liens entre les personnes dans la Figure 1. Par exemple Isis aime Nicolas, c'est-à-dire $aime(Isis, Nicolas)$ mais l'inverse n'est pas vrai. Aussi, Jacquemort et Angel sont amis, soit $ami(Jacquemort, Angel)$.

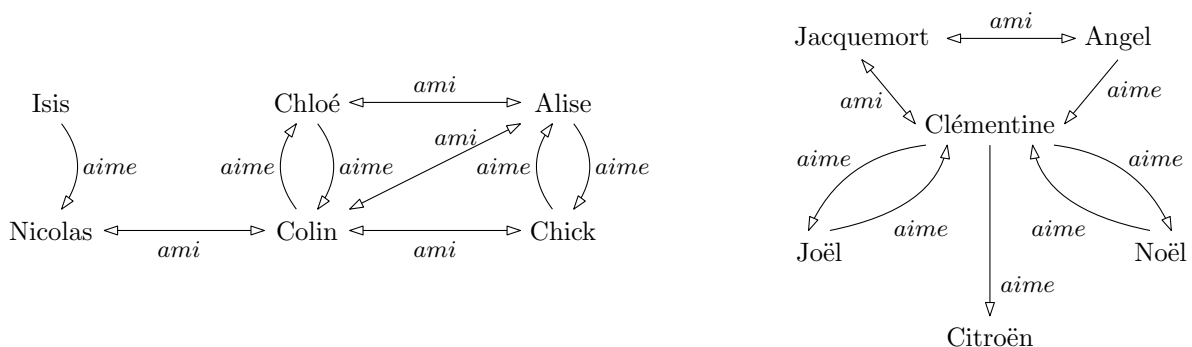


FIGURE 1

Dans cet exemple, nous pouvons par exemple chercher à déterminer les connaissances suivantes :

- les couples possibles. Deux personnes qui s'aiment peuvent former un couple, par exemple Chloé et Colin.
- Les amis communs de deux personnes. Par exemple, le seul ami commun d'Angel et Clémentine est Jacquemort.
- les personnes reliées. Deux personnes sont reliées si l'une aime l'autre, ou si elles sont amies. Par exemple, Noël et Clémentine sont reliés, mais pas Citroën et Noël.

2 Programme : faits et règles

Nous allons créer notre premier programme Prolog, nommé `liens.pl`. Un programme Prolog s'écrit dans n'importe quel éditeur de texte. Dans celui-ci, nous allons d'abord déclarer les *faits* de nos relations d'amitié et d'amour. Par exemple, le fait « Isis aime Nicolas » s'écrit comme suit

```
aime(isis, nicolas). % isis aime nicolas
```

Remarques :

- une ligne se termine *toujours* par un point.
- en Prolog, le nom d'une constante doit *toujours* commencer par une *minuscule*. Les variables commencent par des *majuscules*. Ici, `isis` et `nicolas` sont donc des constantes. Par contre, `X` et `Isis` seront des variables.
- `aime` est un *prédicat* d'arité 2. Les noms de prédicats commencent aussi par une *minuscule*.

Le reste de la relation `aime` s'écrit comme suit :

```
aime(chloe, colin). % chloe aime colin
aime(colin, chloe). % colin aime chloe
aime(chick, alise). % chick aime alise
aime(alise, chick). % alise aime chick
aime(angel, clementine). % angel aime clementine
aime(clementine, joel). % clementine aime joel
aime(joel, clementine). % joel aime clementine
aime(clementine, citroen). % clementine aime citroen
aime(clementine, noel). % clementine aime noel
aime(noel, clementine). % noel aime clementine
```

De manière similaire, nous pouvons maintenant écrire les faits associés à la relation d'amitié. Par exemple, pour énoncé le fait « Chloé et Alise sont amies », on écrit :

```
ami(chloe, alise). % chloe alise sont amies
ami(alise, chloe).
```

Remarques :

- puisque la relation `ami` est symétrique, il faut bien écrire la relation dans les deux sens !

Le reste de la relation suit :

```
% relation d amitie
ami(chloe, alise). % chloe alise sont amies
ami(alise, chloe).
ami(colin, alise). % colin et alise sont amis
ami(alise, colin).
ami(colin, chick). % chick et colin sont amis
ami(chick, colin).
ami(nicolas, colin). % nicolas et colin sont amis
ami(colin, nicolas).
ami(jacquemort, angel). % jacquemort et angel sont amis
ami(angel, jacquemort).
ami(jacquemort, clementine). % jacquemort et clementine sont amis
ami(clementine, jacquemort).
```

Nous avons déclaré tous les faits dont nous disposons. À présent, nous allons nous intéresser aux *règles* nous permettant de déduire de nouvelles connaissances. Une règle a la forme $C \rightarrow T$ (ou $T \leftarrow C$) où C est le *corps* de la règle et T sa *tête*. Commençons par la relation de couple. Nous pouvons l'exprimer en langage logique comme ceci :

$$\forall x, y \quad ((x \neq y) \wedge aime(x, y) \wedge aime(y, x)) \rightarrow couple(x, y)$$

La tête de la règle est $couple(x, y)$ et le corps est $(x \neq y) \wedge aime(x, y) \wedge aime(y, x)$. Cette règle se traduit directement en Prolog :

```
couple(X, Y) :- X \== Y, aime(X, Y), aime(Y, X).
```

Remarques :

- `couple` est un prédicat, comme `aime`.
- la *virgule* sert de *et logique*,
- X et Y sont des *variables* (commencent par des *majuscules*), et la quantification universelle est *implicite*.
- le corps de la règle est $X \neq Y$, `aime(X, Y)`, `aime(Y, X)`, la tête `couple(X, Y)`.
- il ne faut pas oublier le point à la fin de la ligne.

De manière similaire, on peut d'abord décrire de manière logique la règle qui détermine si une personne z est l'ami commun de deux personnes x et y :

$$\forall x, y, z \quad ((x \neq y) \wedge (y \neq z) \wedge (x \neq z) \wedge ami(x, z) \wedge ami(z, y)) \rightarrow ami-commun(z, x, y)$$

Et traduire cette règle en Prolog :

```
ami-commun(Z, X, Y) :- Y \== Z, X \== Z,
ami(X, Z), ami(Z, Y), X \== Y.
```

Remarques :

- il est possible de revenir à la ligne dans une règle.
- puisque la relation d'amitié est symétrique, il est suffisant de tester les prédicats `ami(X, Z)` et `ami(Z, Y)`.

Il nous reste à traduire la règle qui détermine si deux personnes sont reliées :

$$\forall x, y \quad (((x \neq y) \wedge aime(x, y)) \vee ((x \neq y) \wedge aime(y, x)) \vee ((x \neq y) \wedge ami(x, y))) \rightarrow relie(x, y)$$

Ce que l'on traduit en Prolog par :

```
relie(X, Y) :- X \== Y, aime(X, Y).
relie(X, Y) :- X \== Y, aime(Y, X).
relie(X, Y) :- X \== Y, ami(X, Y).
```

Remarques

- définir *plusieurs règles avec la même tête* permet de simuler le *OU logique* : pour que `relie` soit vrai, il faut que la première ligne soit vraie OU la deuxième OU la troisième. Par exemple, `relie(X, Y) :- X == Y, aime(X, Y).` est la traduction de $((x \neq y) \wedge aime(x, y)) \rightarrow relie(x, y)$.
- le prédicat `relie` est fait de ces trois règles.

Nous avons exprimé tous nos faits et règles ! Le fichier `liens.pl` est disponible à la toute fin de ce document. Nous sommes à présents prêts à utiliser l'interpréteur, et à poser des requêtes à notre programme (notre base de connaissance).

3 Requêtes et interpréteur

Nous allons à présent utiliser l'interpréteur `SWI-Prolog` pour charger notre fichier `liens.pl` et effectuer des requêtes dessus. Une fois l'interpréteur ouvert, on peut charger un fichier en allant dans le menu `File` et en cliquant sur `Consult` ... Il suffit ensuite de trouver le fichier à charger dans l'explorateur.

Attention! Si vous changez le code du fichier que vous avez chargé, il faut retourner dans le menu `File` de l'interpréteur, et cliquer sur `reload modified files` pour bien prendre en compte les mises à jour!

Une fois ceci fait, nous pouvons interroger notre fichier :

```
?- aime(colin, chloe).  
true.
```

Cette commande demande s'il est vrai que Colin aime Chloé, ce qui est vrai. Par contre, Colin et Chloé ne sont pas amis, donc nous aurons :

```
?- ami(colin, chloe).  
false.
```

Il est possible de faire des requêtes plus complexes, par exemple trouver les amis de Colin :

```
?- ami(colin, X).  
X = alise ;  
X = chick ;  
X = nicolas.
```

Ici, Prolog va chercher à remplacer la variable `X` par toutes les valeurs qui peuvent rendre le prédicat vrai, c'est l'*unification*. Remarques :

- Prolog va proposer les réponses possible une par une : pour énumérer la prochaine solution, il faut utiliser le point-virgule `;`, et `.` ou `a` pour arrêter.

Essayons à présent d'utiliser nos règles. Nous voulons par exemple savoir quels sont les couples possibles :

```
?- couple(X, Y).  
X = chloe ,  
Y = colin ;  
X = colin ,  
Y = chloe ;  
X = chick ,  
Y = alise ;  
X = alise ,  
Y = chick ;  
X = clementine ,  
Y = joel ;  
X = joel ,  
Y = clementine ;  
X = clementine ,  
Y = noel ;  
X = noel ,  
Y = clementine.
```

Ici Prolog va chercher à rendre `couple(X, Y)` vrai. Pour ce faire, il va devoir satisfaire toutes les conditions nécessaires, telles que nous les avons définies dans le fichier `liens.pl` : `X \== Y`, `aime(X, Y)` et `aime(Y, X)`. Remarquons

que l'on obtient chaque couple deux fois car *X* et *Y* peuvent être intervertis. Ce problème peut être réglé en utilisant la coupe, que nous ne verrons pas ici.

Attention! Prolog évalue les conditions dans l'ordre dans lequel elles sont écrites! C'est l'aspect *procédural* du langage. Ainsi, en fonction de la manière dont les règles sont écrites, l'évaluation peut prendre plus ou moins de temps, voire ne pas fonctionner du tout (boucles infinies)!

Nous pouvons aussi déterminer si des personnes ont des amis en commun :

```
?- ami-commun(Z, angel, clementine).  
Z = jacquemort.
```

Ou si une personne est l'ami commun de deux autres :

```
?- ami-commun(jacquemort, X, Y).  
X = angel,  
Y = clementine ;  
X = clementine,  
Y = angel ;  
false.
```

Dans la prochaine section, nous allons nous intéresser aux *listes* et à la récursivité dans Prolog.

4 Listes

Les listes se déclarent entre crochets `[ ]`. Par exemple `[chloe, colin, angel, noel]` est une liste. La liste vide s'écrit `[]`, une liste contenant un unique élément s'écrit par exemple `[chloe]`. En Prolog les listes sont pensées de manière récursive : une liste est composée d'une *tête*, et d'une autre liste, la *queue*. Dans notre exemple, `chloe` est la tête et `[colin, angel, noel]` la queue. Ainsi, une autre écriture (équivalente) de la liste `[chloe, colin, angel, noel]` est `[chloe | [colin, angel, noel]]`. On a `[chloe | []] = [chloe]`. Nous pouvons à présent écrire des règles pour faire des opérations sur les listes. Par exemple, nous voulons écrire un prédicat qui détermine qu'un élément *x* appartient à une liste *L*. Ce prédicat sera *membre(x, L)*. De manière récursive :

- *x* est dans *L* si *x* est la tête de *L*.
- sinon, *x* est dans *L* s'il est dans la queue de *L*.

Traduisons ces règles en Prolog :

```
% membre(X, L) version 1 : vrai si X est un membre de L  
membre(_, L) :- L = [], false. % L est vide  
membre(X, L) :- L = [X | _], true. % X est la tete de L  
membre(X, L) :- L = [_ | Q], membre(X, Q). % X est ailleurs dans L
```

Le `_` est la *variable non nommée*. Elle sert de « remplissage » lorsque l'on n'a pas besoin d'utiliser la valeur de la variable en question. Notons que le prédicat `membre` peut s'écrire de manière plus compacte :

```
% membre(X, L) version 2 : vrai si X est un membre de L  
membre(X, [X | _]). % X est la tete de L  
membre(X, [_ | L]) :- membre(X, L). % X est dans la queue de L
```

Essayons de faire des requêtes avec ce nouveau prédicat (sans oublier de recharger le fichier dans l'interpréteur) :

```
?- membre(colin, [chloe, colin, angel, noel]).  
true.  
  
?- membre(joel, [chloe, colin, angel, noel]).
```

```

false.

?- membre(X, [chloe, colin, angel, noel]).
X = chloe ;
X = colin ;
X = angel ;
X = noel ;
false.

```

On peut aussi construire un prédicat qui filtre une liste de personne en n'en gardant qu'une sur deux (en gardant la première). Appelons ce prédicat `filtre(L, R)`. De manière récursive :

- si L est vide, R sera vide (rien à filtrer),
- si L n'a qu'un élément, R aura seulement cet élément aussi,
- sinon il faut ajouter à R la tête de L et sauter le deuxième élément.

Ces règles se traduisent ainsi :

```

% filtre(L, R) version 1 : enleve un element sur deux de L, et
% met le reste dans R
filtre(L, R) :- L = [], R = []. % si L est vide, R est vide

% si L a un seul element, R aura seulement cet element
filtre(L, R) :- L = [X], R = [X].

% sinon, on met un element sur deux dans R
filtre(L, R) :- L = [T1 | Q1], Q1 = [_ | Q2], filtre(Q2, R2), R = [T1 | R2].

```

Ou de manière plus compacte :

```

% filtre(L, R) version 2 : enleve un element sur deux de L, et
% met le reste dans R
filtre([], []). % si L est vide, R est vide

% si L a un seul element, R aura seulement cet element
filtre([X], [X]).

% sinon, on met un element sur deux dans R
filtre([X, _ | L], [X | R]) :- filtre(L, R).

```

Remarques :

- Il n'est pas possible de renvoyer de listes en Prolog, il faut donc utiliser un paramètre « *résultat* » qui stockera notre liste finale. L'idée est la suivante : en essayant de rendre le prédicat `filtre(L, R)` vrai, Prolog va être amené à suivre les deux règles énoncées, et donc construire le résultat voulu.
- on peut choisir de mettre autant d'éléments que l'on veut dans la tête de la liste ! Ici, deux.

Quand on applique celui-ci, l'on obtient par exemple :

```

?- filtre([colin, nicolas, clementine, chick, alise, chloe, noel], R).
R = [colin, clementine, alise, noel].

?- filtre([colin, nicolas, clementine, chick, alise, chloe], R).
R = [colin, clementine, alise].

```

Ceci conclue notre bref tour d’horizon sur `Prolog`. Le fichier final `liens.pl` est donné dans la section suivante. Pour plus de détails, voici une petite liste de liens utiles :

- ▶ Learn Prolog Now, <http://learnprolognow.org>
- ▶ Cours de Dominique Pastre, <http://www.normalesup.org/~pastre/prolog/cours.pdf>
- ▶ Wikilivre Prolog, <https://en.wikibooks.org/wiki/Prolog>
- ▶ Documentation SWI-Prolog, <http://www.swi-prolog.org/pldoc/index.html>

5 Fichier final

```
/* Fichier liens.pl */

% relation d amour
aime(isis, nicolas). % isis aime nicolas
aime(chloe, colin). % chloe aime colin
aime(colin, chloe). % colin aime chloe
aime(chick, alise). % chick aime alise
aime(alise, chick). % alise aime chick
aime(angel, clementine). % angel aime clementine
aime(clementine, joel). % clementine aime joel
aime(joel, clementine). % joel aime clementine
aime(clementine, citroen). % clementine aime citroen
aime(clementine, noel). % clementine aime noel
aime(noel, clementine). % noel aime clementine

% relation d amitie
ami(chloe, alise). % chloe alise sont amies
ami(alise, chloe).
ami(colin, alise). % colin et alise sont amis
ami(alise, colin).
ami(colin, chick). % chick et colin sont amis
ami(chick, colin).
ami(nicolas, colin). % nicolas et colin sont amis
ami(colin, nicolas).
ami(jacquemort, angel). % jacquemort et angel sont amis
ami(angel, jacquemort).
ami(jacquemort, clementine). % jacquemort et clementine sont amis
ami(clementine, jacquemort).

% relation de couple
couple(X, Y) :- X \== Y, aime(X, Y), aime(Y, X).

% relation ami-commun
ami-commun(Z, X, Y) :- Y \== Z, X \== Z, ami(X, Z), ami(Z, Y), X \== Y.

% relation relie
relie(X, Y) :- X \== Y, aime(X, Y).
relie(X, Y) :- X \== Y, aime(Y, X).
relie(X, Y) :- X \== Y, ami(X, Y).

% membre(X, L) : vrai si X est un membre de L
membre(X, [X | _]). % X est la tete de L
membre(X, [_ | L]) :- membre(X, L). % X est dans la queue de L

% filtre(L, R) : enleve un element sur deux de L, et met le reste dans R
```

```

filtre([], []). % si L est vide, R est vide

% si L a un seul element, R aura seulement cet element
filtre([X], [X]).

% sinon, on met un element sur deux dans R
filtre([X, _ | L], [X | R]) :- filtre(L, R).

/* memento :
*   - constante --> minuscule
*   - variable --> majuscule
*   - terminer une ligne par un .
*   - predicat(...). est un FAIT
*   - predicat(...) :- r(...), s(...). est une REGLE
*   - dans une regle:
*       . virgule --> ET logique,
*       . retour a la ligne --> OU logique,
*       . not() --> negation
*   - [T | Q] est une liste, T la tete, et Q la queue
*/

```